Bridge Pattern Implementation Report

Introduction

The Bridge pattern separates abstraction from implementation, enabling independent variation. The original implementation (Refactoring Guru - Bridge Pattern Example) uses this pattern to control devices via remotes.

New Functionality

Two new classes are added:

- `SmartTV`: A smart TV capable of browsing the internet.
- `SmartRemote`: A remote with voice control.

Motivation

To demonstrate how the Bridge pattern can accommodate the integration of new features into existing implementations without altering existing structure.

Implementation

SmartTV

```
public class SmartTV extends Device {
    // ...

    public void browseInternet() {
        System.out.println("Browsing the Internet on Smart TV");
    }

    // ...
}
```

SmartRemote

```
public class SmartRemote extends AdvancedRemote {

    public SmartRemote(Device device) {
        super(device);
    }

    public void voiceControl() {
        System.out.println("Remote: Voice control");
    }
}
```

Verification

New functionality is tested by instantiating `SmartTV` and `SmartRemote` objects and invoking their methods.

```
    public static void main(String[] args) {
        testDevice(new Tv());
        testDevice(new Radio());
        testDevice(new SmartTV());
    }

    public static void testDevice(Device device) {
        // ...

        System.out.println("Tests with smart remote.");
        SmartRemote smartRemote = new SmartRemote(device);
        smartRemote.power();
        smartRemote.mute();
        smartRemote.voiceControl();
        if (device instanceof SmartTV) {
            ((SmartTV) device).browseInternet();
        }
        device.printStatus();
    }
```

Execution results:

```
...

Tests with smart remote.
Remote: power toggle
Remote: mute
Remote: voice control
Browsing the Internet on Smart TV
-----------------------------------
| I'm SmartTV set.
| I'm enabled
| Current volume is 0%
| Current channel is 1
-----------------------------------
```

Conclusion

The completed assignment showcases flexibility and extensibility of the Bridge pattern, hence enhances code maintainability, scalability, and adaptability, ensuring that the system remains robust and versatile in accommodating future changes.